

Name: _____

Quiz 1

1. [3 points] Consider the following visual representation of a circular array, as studied in class, having a length of 10, with 7 elements stored currently.

50	81	41				66	73	3	24
----	----	----	--	--	--	----	----	---	----

What is the value of j , the index at which the first element resides: 6

Suppose that `add(2, 99)` was performed. Draw the resulting array in the space below:

50	81	41			66	73	99	3	24
----	----	----	--	--	----	----	----	---	----

Suppose that `remove(6)` was performed. Draw the resulting array in the space below:

50	81					66	73	3	24
----	----	--	--	--	--	----	----	---	----

2. [2 points] Consider a mathematical set¹, S , of numbers; $S = \{66, 73, 3, 24, 50, 81, 41\}$. S is stored two ways; as a **USet** and as an **SSet**.

What will `find(30)` return when called on the **USet**: nil

What will `find(30)` return when called on the **SSet**: 41

3. [4 points] Consider an **SSet** based implementation of the following Priority Queue (PQ) interface:

Priority Queue Interface

`add(p, x)` : add element x with associated priority p into the queue
`remove()` : remove the element with lowest priority from the queue

The elements of the PQ are stored as pairs, (p, x) , inside the **SSet**, where p corresponds to the priority, a real number ranging between 0 and 1 exclusive, and x corresponds to the element stored. The equality, lesser than and greater than operators are defined in the following way over the **SSet**:

Operators

$=$: $(p, x) = (q, y)$ iff p is equal in value to q .
 $<$: $(p, x) < (q, y)$ iff p is lesser in value than q .
 $>$: $(p, x) > (q, y)$ iff p is greater in value than q .

¹ Note that order doesn't matter in the standard mathematical set

Name: _____

Using only the operations of **SSet** interface, write code for the following PQ operations:

- a. `add(p, x) : add((p, x))`
- b. `remove() : remove(find((0, _)))`

4. [6 points] In this question, the underlying implementation uses a circular array.

In the `add(i, x)` operation studied in class, there is a condition over the value of index i , namely “if $i \geq n/2$...”. Suppose that the condition was modified to be “if $i \geq n/3$...”, i.e. instead of checking whether the index lies in the second half or not, the algorithm now checks whether the index lies in the last two-thirds of the array or not. Analyze whether the time complexities of the following operations would change, and if so, how:

- a. Time complexity of `add(x)` from the **FIFO Queue** interface.

doesn't change, adding at the end of the queue, so it is still $O(1)$

- b. Time complexity of `pop()` from the **Stack** interface.

doesn't change, pop doesn't utilize the add operation

- c. Time complexity of `add(i, x)` from the **List** interface.

changes, it is represented by the following piecewise function:

$$\begin{aligned} f(n, i) &= O(i) && \text{if } i < \frac{1}{3}n \\ f(n, i) &= O(n-i) && \text{otherwise} \end{aligned}$$